Отчет по лабораторной работе №1:

Полнодоступная двухсервисная модель Эрланга с одинаковыми интенсивностями обслуживания

Дисциплина: Основы проектирования сетей и систем телекоммуникаций

Выполнил: Кхари Жекка Кализая Арсе, НПИбд-01-23

Содержание

1 алгоритм расчета вероятностных характеристик модели **3**

2 Численный анализ **4**

3 исходный код **10**
3.1 lab02_task.py 10
3.2 test.py 12

1 алгоритм расчета вероятностных характеристик модели

1. читать/вводить/загрузить в памяти исходные данные
2. рассчитать интенсивность предложенной нагрузки ρ_1 ρ_2
3. рассчитать первую вероятность P_0
4. рассчитать все вероятности P_n (с P_0 до P_c)
5. проверять: $\sum_{n=0}^{C} p_n = 1$
6. рассчитать вероятность блокировки E_1:
   $E_1 = p_C$
7. рассчитать вероятность блокировки E_2:

   $E_2 = \sum_{n=g}^{C} p_n$
8. рассчитать средное число заявок
9. показывать результаты

2 Численный анализ

Для расчёта основных вероятностных характеристик модели были взяты следующие параметры.

$$C = 10, g = 3, \lambda_1 = 20, \lambda_2 = 30, \mu = 0.5 \tag{2.1}$$

Код написане на языке Python и выполнен в среде Visual Studio Code.

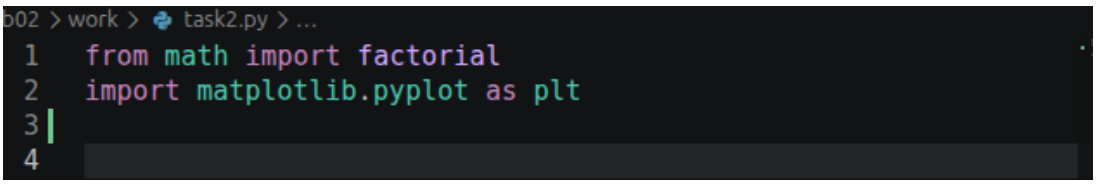


Рисунок 2.1: импортируем библиотеки

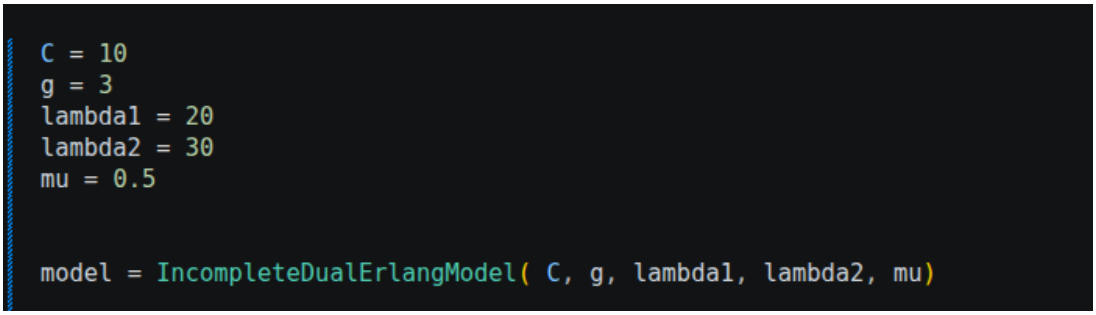


Рисунок 2.2: задаем начальные значения объекту model в файл test.py



Рисунок 2.3: функция инициализации класса IncompleteDualErlang-Model

```
def calculate_P0(self):
    self.P0 = 0
    suma1 = 0
    suma2 = 0
    for n in range(1,self.reservedCapacity+1):
        suma1 = suma1+ ((self.rho1 + self.rho2 )**n)/ factorial(n)

    for n in range(self.reservedCapacity+1,self.Capacity+1):
        suma2 = suma2 + (((self.rho1 + self.rho2)**self.reservedCapacity * self.rho1**(n-self.reservedCapacity))/factorial(n))


    self.P0= (1 + suma1 + suma2)**-1
```

Рисунок 2.4: функция рассчета вероятности P_0

```
def calculateArrayOfProbabilities(self):

    self.arrayOfProbabilities = []
    self.arrayOfProbabilities.append(self.P0)


    for n in range(1,self.reservedCapacity+1):
        self.arrayOfProbabilities.append(
            self.P0 * ((self.rho1 + self.rho2 )**n)/factorial(n)
        )

    for n in range(self.reservedCapacity+1, self.Capacity+1):
        self.arrayOfProbabilities.append(
            (self.P0 * ((self.rho1 + self.rho2)**self.reservedCapacity) * self.rho1**(n - self.reservedCapacity) )/ factorial(n)
        )
```

Рисунок 2.5: Функция рассчета стационарного распределения вероятностей состояний системы

```
    self.calculate_P0()
    self.calculateArrayOfProbabilities()

    self.probabilityOfBlockingFirstType = self.arrayOfProbabilities[-1]
    self.probabilityOfBlockingSecondType = sum( self.arrayOfProbabilities[reservedCapacity:])
```

Рисунок 2.6: рассчета значения Вероятность блокировки по времени E_1 запроса на предоставление услуги 1-го типа

```
    self.calculate_P0()
    self.calculateArrayOfProbabilities()

    self.probabilityOfBlockingFirstType = self.arrayOfProbabilities[-1]
    self.probabilityOfBlockingSecondType = sum( self.arrayOfProbabilities[reservedCapacity:])
```

Рисунок 2.7: рассчета значения Вероятность блокировки по времени E_2 запроса на предоставление услуги 2-го типа

```
    self.calculate_P0()
    self.calculateArrayOfProbabilities()

    self.probabilityOfBlockingFirstType = self.arrayOfProbabilities[-1]
    self.probabilityOfBlockingSecondType = sum( self.arrayOfProbabilities[reservedCapacity:])


    self.averageRequestServerd =  sum( [x *y for x ,y in zip(range(0,self.Capacity+1), self.arrayOfProbabilities  ) ]  )
```

Рисунок 2.8: рассчет среднего числа запросов в системе

```
def printTask2(self):
    print(f"значение разпределения вероятностей")
    for i in range(len(self.arrayOfProbabilities)):
        print(f"P[{i}]: {self.arrayOfProbabilities[i]}")

    print(f"значение вероятностей блокировка lambda1: {self.probabilityOfBlockingFirstType}")
    print(f"значение вероятностей блокировка lambda2: {self.probabilityOfBlockingSecondType}")
    print(f"значение среднего числа заявок: {self.averageRequestServerd}")
```

Рисунок 2.9: функция вывода вероятностных характеристик модели

```
def drawTask3(self):
    y = []
    for i in range(int(self.Capacity)):
        model = IncompleteDualErlangModel(
            self.Capacity,
            self.reservedCapacity,
            i,
            self.arrival_rate_2,
            self.u
            )

        y.append(model.probabilityOfBlockingFirstType)

    plt.figure(figsize=(10,5))
    plt.bar(range(int(self.Capacity)), y)
    plt.title("график зависимости вероятности блокировки от \n" \
    "интенсивности предложенной нагрузки, создаваемой запросами первого типа")
    plt.xlabel("интенсивность поступления запросов на предоставление услуги 1")
    plt.ylabel("вероятность блокировки")
    plt.show()
```

Рисунок 2.10: функция построения графика зависимости вероятности блокировки от интенсивности предложенной нагрузки, создаваемой запросами первого типа

```python
def drawTask4(self):
    step = 1
    y = []
    for i in range(0,self.Capacity,step):
        increment_factor = i/self.Capacity
        model = IncompleteDualErlangModel(
            self.Capacity,
            self.reservedCapacity,
            self.arrival_rate_1*increment_factor,
            self.arrival_rate_2*increment_factor,
            self.u
            )

        y.append(model.probabilityOfBlockingFirstType)

    plt.figure(figsize=(10,5))
    plt.bar(range(len(y)), y)
    plt.title("график зависимости среднего числа обслуживаемых запросов от \n" \
    "интенсивности поступления запросов на предоставление услуги")
    plt.xlabel("интенсивность поступления запросов на предоставление услуги 1")
    plt.ylabel("вероятность блокировки")
    plt.show()
```

Рисунок 2.11: функция Построения графика зависимости среднего числа обслуживаемых запросов от интенсивности поступления запросов на предоставление услуги.

```python
if __name__ == "__main__":
    print("лабораторная работа №2")
    print("Неполнодоступная двухсервисная модель Эрланга с одинаковыми")

    print("введите переменные")

    C = int(input("C: "))
    g = int(input("g: "))
    lambda1 = float(input("lambda1: "))
    lambda2 = float(input("lambda2: "))
    u = float(input("u: "))

    model = IncompleteDualErlangModel( C, g, lambda1,  lambda2, u)
    model.printTask2()
    model.drawTask3()
    model.drawTask4()
```

Рисунок 2.12: ввод данных вручную

```
ecciónDeRedes/lab02/work/test.py
значение разпределения вероятностей
P[0]: 1.6781601203241756e-11
P[1]: 1.6781601203241756e-09
P[2]: 8.390800601620878e-08
P[3]: 2.796933533873626e-06
P[4]: 2.796933533873626e-05
P[5]: 0.00022375468270989006
P[6]: 0.0014916978847326007
P[7]: 0.008523987912757717
P[8]: 0.04261993956378859
P[9]: 0.18942195361683817
P[10]: 0.7576878144673526
```

Рисунок 2.13: Стационарное распределение вероятностей системы для всех n от 0 до C

```
значение вероятностей блокировка lambda1: 0.7576878144673526
значение вероятностей блокировка lambda2: 0.9999999143970522
```

Рисунок 2.14: значения вероятностей блокировки

```
значение среднего числа заявок: 9.692492557482755
```

Рисунок 2.15: значение среднего числа заявоко

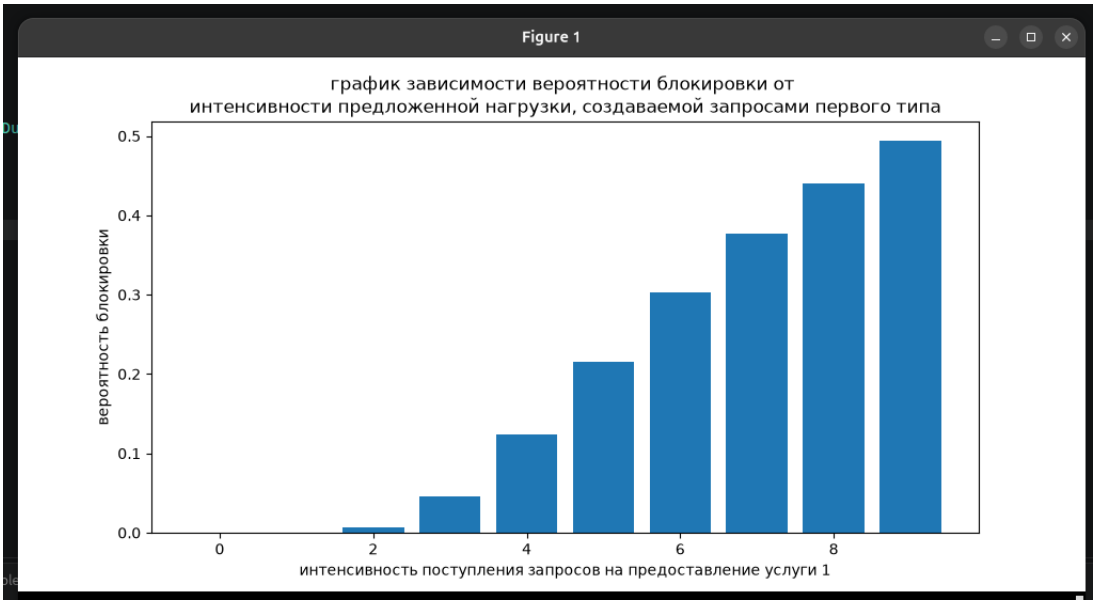


Рисунок 2.16: график зависимости вероятности блокировки от интенсивности предложенной нагрузки, создаваемой запросами первого типа

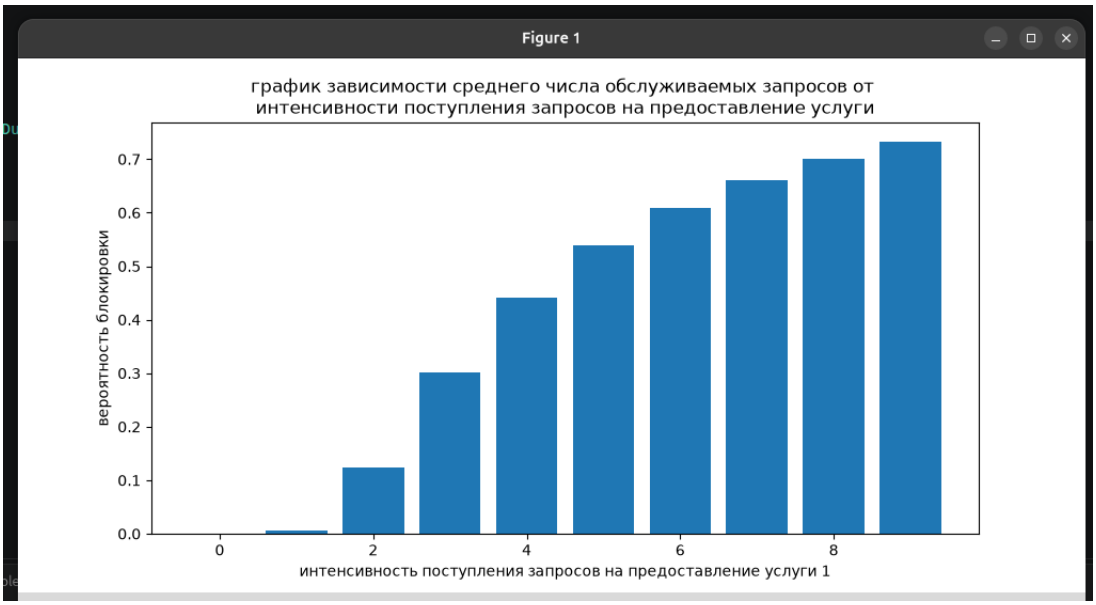


Рисунок 2.17: график зависимости среднего числа обслуживаемых запросов от интенсивности поступления запросов на предоставление услуги

3 исходный код

3.1 lab02_task.py

```
from math import factorial
import matplotlib.pyplot as plt


class IncompleteDualErlangModel:

    def __init__(
            self,
            Capacity:int,
            reservedCapacity:int,
            arrival_rate_1:float,
            arrival_rate_2:float,
            u:float
            ):

        self.Capacity = Capacity
        self.reservedCapacity = reservedCapacity
        self.arrival_rate_1 = arrival_rate_1
        self.arrival_rate_2 = arrival_rate_2
        self.u = u

        self.rho1 = arrival_rate_1/u
        self.rho2 = arrival_rate_2/u

        self.calculate_P0()
        self.calculateArrayOfProbabilities()

        self.probabilityOfBlockingFirstType = self.arrayOfProbabilities[-1]
        self.probabilityOfBlockingSecondType = sum(
        ↪  self.arrayOfProbabilities[reservedCapacity:])


        self.averageRequestServerd =  sum( [x *y for x ,y in
        ↪  zip(range(0,self.Capacity+1), self.arrayOfProbabilities  ) ]   )



    def calculate_P0(self):
        self.P0 = 0
        suma1 = 0
        suma2 = 0
        for n in range(1,self.reservedCapacity+1):
            suma1 = suma1+ ((self.rho1 + self.rho2 )**n)/ factorial(n)
```

```
        for n in range(self.reservedCapacity+1,self.Capacity+1):
            suma2 = suma2 + (((self.rho1 + self.rho2)**self.reservedCapacity *
↪  self.rho1**(n-self.reservedCapacity))/factorial(n))


        self.P0= (1 + suma1 + suma2)**-1



    def calculateArrayOfProbabilities(self):

        self.arrayOfProbabilities = []
        self.arrayOfProbabilities.append(self.P0)


        for n in range(1,self.reservedCapacity+1):
            self.arrayOfProbabilities.append(
                self.P0 * ((self.rho1 + self.rho2 )**n)/factorial(n)
            )

        for n in range(self.reservedCapacity+1, self.Capacity+1):
            self.arrayOfProbabilities.append(
                (self.P0 * ((self.rho1 + self.rho2)**self.reservedCapacity) *
↪  self.rho1**(n - self.reservedCapacity) )/ factorial(n)
            )


    def printTask2(self):
        print(f"значение разпределения вероятностей")
        for i in range(len(self.arrayOfProbabilities)):
            print(f"P[{i}]: {self.arrayOfProbabilities[i]}")

        print(f"значение вероятностей блокировка lambda1:
         ↪  {self.probabilityOfBlockingFirstType}")
        print(f"значение вероятностей блокировка lambda2:
         ↪  {self.probabilityOfBlockingSecondType}")
        print(f"значение среднего числа заявок: {self.averageRequestServerd}")
         ↪


    def drawTask3(self):
        y = []
        for i in range(int(self.Capacity)):
            model = IncompleteDualErlangModel(
                self.Capacity,
                self.reservedCapacity,
                i,
                self.arrival_rate_2,
                self.u
                )

            y.append(model.probabilityOfBlockingFirstType)

        plt.figure(figsize=(10,5))
        plt.bar(range(int(self.Capacity)), y)
        plt.title("график зависимости вероятности блокировки от \n" \
```

```
        "интенсивности предложенной нагрузки, создаваемой запросами первого
        ↪  типа")
        plt.xlabel("интенсивность поступления запросов на предоставление услуги
↪  1")
        plt.ylabel("вероятность блокировки")
        plt.show()

    def drawTask4(self):
        step = 1
        y = []
        for i in range(0,self.Capacity,step):
            increment_factor = i/self.Capacity
            model = IncompleteDualErlangModel(
                self.Capacity,
                self.reservedCapacity,
                self.arrival_rate_1*increment_factor,
                self.arrival_rate_2*increment_factor,
                self.u
                )

            y.append(model.probabilityOfBlockingFirstType)

        plt.figure(figsize=(10,5))
        plt.bar(range(len(y)), y)
        plt.title("график зависимости среднего числа обслуживаемых запросов от
↪  \n" \
        "интенсивности поступления запросов на предоставление услуги")
        plt.xlabel("интенсивность поступления запросов на предоставление услуги
↪  1")
        plt.ylabel("вероятность блокировки")
        plt.show()


if __name__ == "__main__":
    print("лабораторная работа №2")
    print("Неполнодоступная двухсервисная модель Эрланга с одинаковыми")

    print("введите переменные")

    C = int(input("C: "))
    g = int(input("g: "))
    lambda1 = float(input("lambda1: "))
    lambda2 = float(input("lambda2: "))
    u = float(input("u: "))

    model = IncompleteDualErlangModel( C, g, lambda1,  lambda2, u)
    model.printTask2()
    model.drawTask3()
    model.drawTask4()
```

3.2 test.py

```
from lab02.work.lab02_task import IncompleteDualErlangModel
```

```
C = 10
g = 3
lambda1 = 20
lambda2 = 30
mu = 0.5


model = IncompleteDualErlangModel( C, g, lambda1, lambda2, mu)
model.printTask2()
model.drawTask3()
model.drawTask4()
```